SDitH v3.0 Lean formalization: paper-to-theorem report

Oct 14, 2026 · @Xingyu

What the project is

The project is a Lean 4 / Mathlib development around the SDitH v3.0 signature submission (the PDF dated 2026-08-31, SHA-256 `22dfed06…01c83`). It has three layers: a typed specification of the submission's 21 algorithms and primitives (`SDitH/Spec/`), machine-checked refutations of several literal source claims (`SDitH/Final.lean`), and a composed EUF-CMA reduction for an explicitly named candidate scheme in an ideal hash-and-cipher model (`SDitH/Proofs/`).

The rules the development is built under decide what "corresponds to a Lean theorem" can mean:

- No `sorry`, `admit`, custom axioms, `native_decide` or `unsafe`. Every declaration depends only on `propext`, `Classical.choice` and `Quot.sound`; the validator `scripts/check.py` audits this over every project declaration.
- Every cryptographic or model assumption is an explicit hypothesis of the theorem that uses it, written as a named definition. Nothing is asserted as an axiom, and no unproved step is hidden under an assumption's name.
- The literal source is specified as written. Where a literal claim is false, the refutation is kept and a corrected reading is introduced under a separate `Candidate…` name. Nothing is silently repaired.
- Proof settings are untouched: no heartbeat, recursion or trust option is raised anywhere.

The last full validation (continuation 335) built 3,753 jobs and audited 19,149 declarations, every transitive axiom allowlisted. To reproduce it:

```
lake build SDitH
python3 scripts/check.py
```

The rest of this report reads the submission's security argument top-down and names, for each paper-level step, the Lean theorem that carries it, the hypotheses that step still needs, and where a step is not covered at all. Module names are relative to `SDitH.Proofs` unless written in full.

The security claim on paper and in Lean

On paper, the submission's security statement is: in the random-oracle and ideal-cipher model, any forger against SDitH with a bounded number of signing and oracle queries can be turned into a solver for the underlying syndrome-decoding instance, up to explicit statistical terms. In Lean this is one theorem with two forms:

Paper object	Lean statement	Scope
EUF-CMA bound for the scheme, generic cipher layouts	`SeedTermDischarge.forgery_le_final_hard_C`	Any layout of the settings' type, with layout injectivity and fiber bounds as hypotheses
The same at the submitted full layout	`FullLayoutFinal.forgery_le_final_full`	Categories I and V; the seven layout hypotheses are proved and the fiber constants are 1

The shape of the conclusion is

$$\left(\mathrm{Adv}^{\text{EUF-CMA}}_{\mathcal{A}}(\mathrm{fuel}+1) - 2^{-2\lambda}\right)^2 \;\le\; (\mathrm{fuel}+1)^2 \cdot \Bigg(\sum_{\text{events}} \left(\varepsilon_{\text{event}} + \varepsilon_{\text{sim}}\right) + \varepsilon_{\text{slope}} + \varepsilon_{\text{coll}} + \varepsilon_{\text{fork}} + \varepsilon_{SD}(\mathrm{fuel}) + 2^{-2\lambda} \Bigg)$$

where the left side is `advantage (scheme (CandidateScheme.components S V …)) adversary (fuel + 1)`, the forger's winning probability in the bounded adaptive chosen-message game `Spec.SignatureSecurity.experiment`, minus the digest-guessing term `digestInv lambda`. The square and the factor (fuel+1)² are the forking lemma's shape. Section 4 lists every summand with its Lean name.

Three things about the Lean side differ from a paper statement and should be read before anything else:

- **The scheme is the candidate scheme, not the literal one.** `CandidateScheme`, `CandidateVerifier` and `CandidateBAVC` are the named corrected readings of Algorithms 3, 4, 9 and 12. The literal routines cannot be the object of a security theorem because the literal PIOP check and the literal reconstruction reject honest inputs. The three theorems of `SDitH/Final.lean` certify exactly those defects: `literal_piop_not_complete` (an honest Category-I input with a nonzero challenge is

rejected), `table3_dimension_mismatch` (Table 3's tensor dimensions fail in all three categories) and `commitment_discrepancies` (a Category-I BAVC opening at a hidden index is always rejected by the literal bound, and the signer's and verifier's `Hashlines` inputs differ whenever the witness correction is nonempty).

- **The budget counts oracle actions, not time.** `fuel` is the number of game steps and `bound` the oracle budget per phase. There is no machine model, so the statement is not a concrete-time bound.
- **The hardness term is on the fixed public instance.** `εSD fuel` comes from the named hypothesis `InstanceHard` on the key generation's instance `instanceOf S.H' S.y`. Relating it to a distribution over instances (the paper's "random SD instance") is an open obligation; the planted-distribution definition `SDitH.Assumptions.SDHardness` exists but is consumed by no theorem.

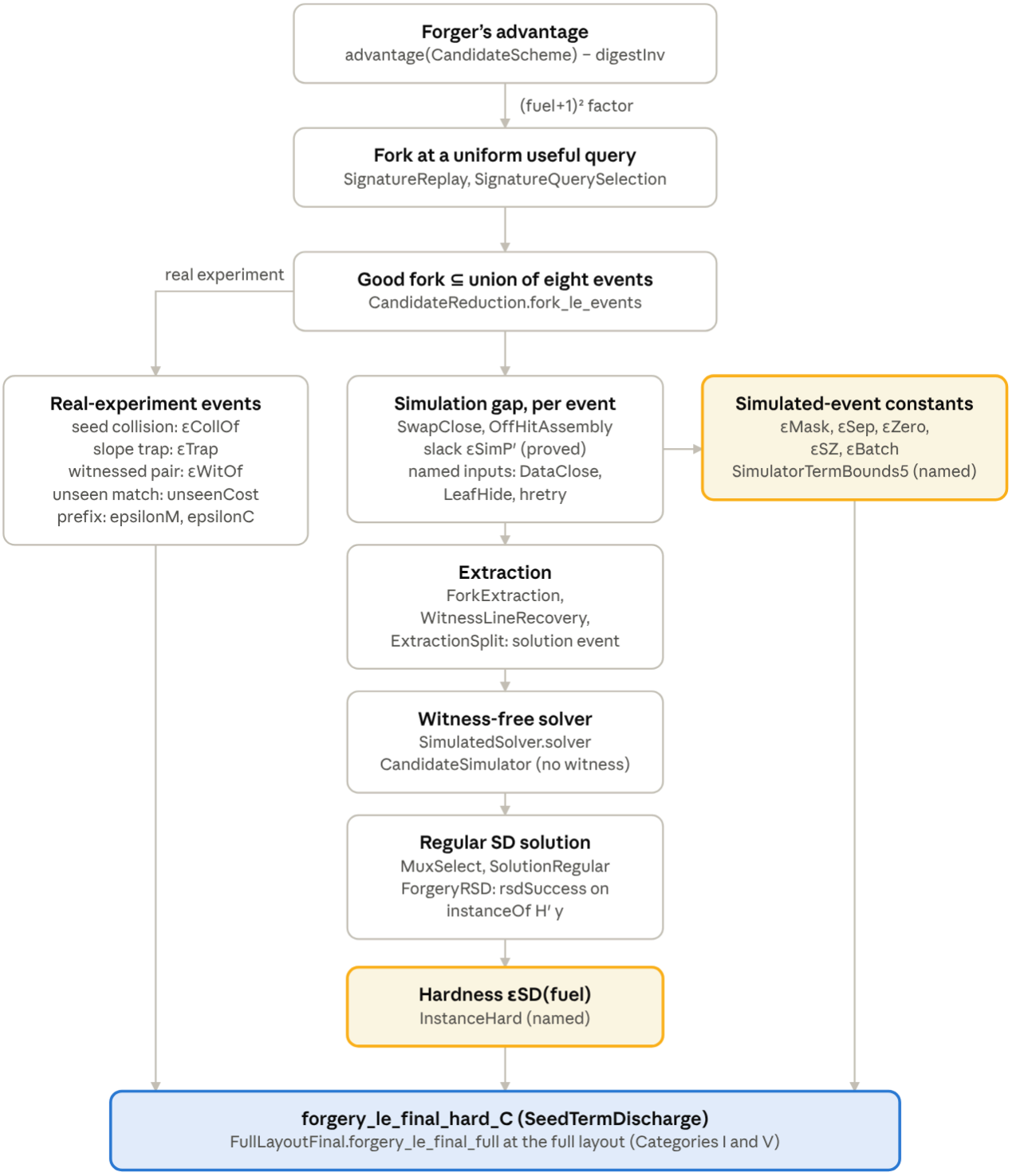
Everything else the paper claims about security either appears as a proved lemma in the chain (section 3), as a named hypothesis (section 5) or is not covered (section 7).

The reduction step by step

The paper's argument has the usual shape for a Fiat-Shamir signature: fork the forger at the hash query it wins on, extract a witness from the two accepted transcripts, replace honest signing by a witness-free simulator, and hand the witness to a syndrome-decoding solver. Each hop is one or a few theorems; the chain below runs top-down from the forger's advantage to the hardness term. Theorem names are `SDitH.Proofs.<Module>.<name>`.

Ten theorems take the forger's advantage to the hardness term

Tinted boxes are named assumptions; every other box is proved.



reduction chain · 10 theorems, 5 named inputs

The middle column is the chain of `forgery_le_*` theorems; the left column's events are bounded in the real experiment and never pass through the simulator, and the right column's are the five constants left to a hypothesis.

1. **Game and replay (paper: the EUF-CMA experiment and the forking lemma).** The game is `Spec.SignatureSecurity.experiment`, a bounded adaptive chosen-message game over a shared lazy hash table and a sampled ideal-cipher family; `advantage` is its winning probability. A forgery that never queried its final hash input wins with probability at most 1/|Digest| (`SignatureSecurity.advantage_le_present_plus_guess`), which is the subtracted `digestInv` term. `Spec.SignatureReplay` defines the cut at a chosen hash action, the fresh second branch and the fork experiment `forkExperiment`; `SignatureQuerySelection.selected_experiment_eq` shows that selecting a uniformly random useful query and replaying there gives the squared-advantage shape, and `SignatureReplay.fresh_branch_answer_uniform` that the second branch's answer is fresh.
2. **Fork outcome as a union of named events (paper: "unless a bad event happens, the two transcripts extract").** `CandidateReduction.fork_le_events` and `ForgeryReduction.fork_le_union` / `forgery_le_events`: a winning fork without the charged prefix events (mention, collision, fixed target: `SharedMentionBound.epsilonM`, `SharedCollisionBound.epsilonC`, `SharedTargetBound`) is one of eight Boolean events on the fork result: mask-after-fork, non-separation, seed collision, slope-game win, zero challenge, Schwartz-Zippel, batching, or extraction. The bound is the sum of their probabilities plus the charged terms.
3. **Deterministic extraction (paper: special soundness of the BLC commitment and the PIOP).** `ForkExtraction.fork_extracts`: two accepted transcripts at the same hash input with different hidden indices give a recorded seed collision, a slope-game win, or a common slope. `WitnessLineRecovery.extract_witness_coordinate` reads the witness coordinate from two line evaluations at distinct merged challenges. `ExtractionSplit.extracts_split` splits the extraction event into the batching event and the solution event (zero mux residual and unitary selectors), using the exact constant-term identity of the PIOP composition.
4. **The witness-free solver (paper: "the reduction simulates signatures without the secret key").** `SimulatedSolver` defines `solver`, which runs the fork experiment and outputs the extracted witness; `solverSuccess_ge` shows the solution event's probability is at most the solver's success. `Spec.CandidateSimulator` is the named simulator; `CandidateSimulator.sign_witness_free` proves its signing never reads the witness half of the key, and `forgery_le_simulator` restates the bound on the simulated scheme with one simulation-gap hypothesis per event.
5. **Simulation gap (paper: honest-verifier zero knowledge of the commitment-and-opening phase in the ROM/ICM).** This is the largest part of the development (roughly continuations 102 to 334). The honest signing is unfolded into the commitment run, the

PIOP query and the opening; hybrids over run, cut, continuation, branch, fork and event experiments are taken in mean form over the posterior cipher family (`SimulationGapMean2.forgery_le_mean`, `SimulationGapCipher`, `SimulationGapCont`); the input-present and row-stored terms are bounded through the prefix's mention steps and the fresh-parent-key chain of the seed tree (`RowBoundExplicit.forgery_le_row`, `TouchBound`, `HitTerm`); and the witness swap is reduced to two data-level hypotheses, `DataClose` and `LeafHide`, through the hidden-leaf tuple and the off-hit chain (`OffHitClose`, `OffHitAssembly`, `LeafTermBound`). The assembled per-event gap is `SwapClose.forgery_le_final_close`, with slack `εSimP'`.

6. **Events bounded in the real experiment instead of simulated (paper: birthday bounds on seed commitments and on the consistency check).** The seed-collision event is bounded by the explicit pair cost `εCollOf` (`SeedCollisionBound.seedCollision_le`), so its simulated copy is not needed (`SeedTermDischarge`). The slope-game win is covered by three events: the mask trap (`SlopeTrapBound.slopeTrap_le`, slack `εTrap`), the witnessed pair (`ForkWitnessedBound.witnessedFork_le`, slack `εWitOf`) and the unseen match (`ForkUnseenBound.fork_unseen_bound`, slack `unseenCost`); `SlopeChainReduction` substitutes them into the skeleton.
7. **Named simulator constants (paper: the remaining statistical terms of the simulation).** Five events of the witness-free simulator are not discharged and are carried as named constants in `SeedTermDischarge.SimulatorTermBounds5`: mask-after-fork, non-separation, zero challenge, Schwartz-Zippel and batching. Section 5 says why.
8. **From the extracted witness to syndrome decoding (paper: "the extracted witness is a regular SD solution").** `MuxSelect.muxTree_select`: with unitary binary selectors the mux tree reads one leaf per chunk. `SolutionRegular.isSolution_sd`: a witness with zero mux residual and unitary selectors gives a weight-w vector solving the systematic instance $[I \mid H']\cdot x = y$ of the key generation (`Spec.KeyGeneration.instanceOf`). `ForgeryRSD.solverSuccess_le_rsd` turns the solver's success into `rsdSuccess`, the probability of returning such a solution, and `forgery_le_final_rsd` restates the bound with it.
9. **The hardness assumption.** `InstanceHardness.InstanceHard εSD` bounds `rsdSuccess` of every budgeted adversary's solver by `εSD fuel`; `forgery_le_final_hard` and its regenerated form `SeedTermDischarge.forgery_le_final_hard_C` are the final generic statement.
10. **The submitted layout.** `FullLayoutFinal` proves the seven cipher-layout hypotheses at `fullLayout lambda` (input, key and output maps injective, so all fiber constants are 1) and instantiates the generic statement as `forgery_le_final_full`.

The chain is linear: every `forgery_le_*` theorem is proved from the previous one by substituting a bound for one summand, and the final statement's proof is a single `gcongr` over the sum. That is also why a change of decision on any one hypothesis is a local proof project and leaves the rest untouched.

The terms of the bound

The right-hand side of `forgery_le_final_full` is (fuel+1)² times a sum. The table lists every summand, the paper-level quantity it stands for, and the Lean name that produces it. "Proved" means the term is an explicit expression in λ, τ, κ, the budgets and the named constants, with a machine-checked bound; "named" means it is a free constant fixed by a hypothesis.

Summand in Lean	Paper quantity	Status	Produced by
`εMask`, `εSep`, `εZero`, `εSZ`, `εBatch`	Probability, in the simulated game, of: a mask input first queried after the fork; hidden indices not separated at some execution; a zero merged challenge; a nonzero difference polynomial vanishing at the challenge; the batching challenge cancelling a nonzero residual	named	`SeedTermDischarge.SimulatorTermBounds5`

Summand in Lean	Paper quantity	Status	Produced by
`εSimP' 1 1 1 q εR εTouch (εHit + εOff) fuel` (once per event)	Simulation gap: distance between the honest signer and the witness-free simulator on one event, over 3(fuel+1)+2 signing calls	proved slack	`SwapClose.forgery_le_final_close`; per-call cost `SwapClose.εCall` with the row bound `rowBound`; touching term `TouchBound.εTouch`; hit term `HitTerm.εHit`; off-hit term `OffHitClose.εOff`; leaf term `LeafTermBound.εLeafOf`
`εR` inside `εSimP'`	Probability that Algorithm 11′s retry does not return at the digest	named	hypothesis `hretry`
`δ` inside `εOff`	Distance of honest and fake signing data at uniform hidden blocks	named	`OffHitAssembly.DataClose`
`δLeaf` inside `εLeafOf`	Probability that a continuation records a replaced hidden block′s key	named	`LeafTermBound.LeafHide`

Summand in Lean	Paper quantity	Status	Produced by
`εCollOf lambda tau kappa 1 fuel`	Birthday term for a seed collision across the two branches: 3(fuel+1) calls times one call's pair cost	proved	`SeedCollisionBound.seedCollision_le`, `εCollOf`
`(fuel+1) · (εTrap + εTrap)`	Mask trap of the slope-consistency game: τ² · budget · 2^-s per branch	proved	`SlopeTrapBound.slopeTrap_le`, `εTrap`
`εWitOf lambda tau kappa fuel`	Witnessed seed pair in the union of the two branches' cipher histories	proved	`ForkWitnessedBound.witnessedFork_le`, `εWitOf`
`unseenCost lambda tau kappa q`	Unqueried seed matching both halves of a recorded commitment: τ·2^κ·2^λ / ((2^λ - q) (2^λ - q - 1))	proved	`ForkUnseenBound.fork_unseen_bound`, `SharedSlopeBound.unseenCost`
`εSD fuel`	Hardness of the key generation's instance	named	`InstanceHardness.InstanceHard` over `ForgeryRSD.rsdSuccess`

Summand in Lean	Paper quantity	Status	Produced by
`(fuel+1) · 2 · epsilonM lambda tau bound (…)`	Mention event of the fork prefix: a fresh digest mentioned by an earlier input, budget · (n + budget) · (τ+2) /	Digest	
`(fuel+1) · 2 · epsilonC lambda bound (…)`	Digest collision in the prefix, budget · (n + budget) /	Digest	
`2 · ((fuel+1)·(2 · bound · digestInv) + …)`	Fixed-target and mention events along the two branches	proved	`SharedTargetBound.runProgram_target_bound`
`digestInv lambda` (inside the sum and subtracted on the left)	Guessing the final hash digest, 1/	Digest	= 2^-2λ

The generic statement `forgery_le_final_hard_C` has the same shape with fiber constants `c`, `cKey`, `cOut`, `cOutV` in place of the 1s and the layout facts as hypotheses.

The size conditions on the right-hand side are hypotheses, not terms: 26 ⩽ λ, 4·τ·2^κ ⩽ 2^24, 2 ⩽ τ·2^κ, and the two counts `hNb`, `hNR` saying the budget-dependent number of cipher calls stays below 2^λ. The budget `q` is any number at least the commitment budget times 3(fuel+1)+2 plus the pre-commitment budget. The Lean statement takes them as hypotheses; whether they hold at a chosen parameter set and fuel is checked by the reader, not proved here.

Named assumptions

Every assumption is a named `def … : Prop` and enters the final theorem as an explicit hypothesis. None is an axiom; the validator confirms the project uses only the three standard axioms. The decision taken earlier in this project is that these stay as cryptographic assumptions rather than being derived from an idealization of the PRG, the challenge expansion or the batching expansion, each of which is a deterministic parameter in the model.

Hypothesis	Lean definition	What it says informally	Why it is not proved
Data closeness	`OffHitAssembly.DataClose S publicKey B δ`	For every witness, fake witness, message and world satisfying the counted invariant, and every bounded functional of the signature and world: with the hidden leaves' blocks replaced by a uniform tuple, the honest signing's sliced data is within δ of the fake signing's, over uniform salt, root and digest. This is honest-verifier zero knowledge of the commitment data at uniform hidden blocks.	The data-level step is proved for a uniform hidden vector (`HonestViewUniform` , continuation 114); going from a uniform seed to a uniform expansion needs the PRG idealized (`PRGUniform` names that predicate, continuation 115).
Leaf hiding	`LeafTermBound.LeafHide S publicKey B δLeaf`	With the tuple's leaves replaced by uniform blocks, a continuation of the game records a replaced block's key with probability at most δLeaf on average.	Same dependence on the PRG.

Hypothesis	Lean definition	What it says informally	Why it is not proved
Retry bound	`hretry` (inline hypothesis of the final theorems)	Under the counted invariant, the candidate retry of Algorithm 11 returns at the digest except with probability εR.	The grinding and nonzero-challenge tests read the challenge expansion, which is deterministic in the model.
Simulator event bounds	`SeedTermDischarge.SimulatorTermBounds5 … εMask εSep εZero εSZ εBatch`	Five events of the witness-free simulator's fork experiment have probability at most the named constants: `maskAfter`, `notSeparated V`, `zeroChallenge V encode`, `schwartzZippel S V encode`, `batchingBad S V`.	`εMask` is the hypothesis term of decision 1. The other four would be small-support counts per attempt if the challenge and batching expansions were uniform; they are deterministic in the model.
Instance hardness	`InstanceHardness.InstanceHard S V verifyBudget publicKey sampleWitness εSD`	For every adversary state type, adversary and fuel, the witness-free solver's probability of returning a weight-w solution of `instanceOf S.H' S.y` is at most `εSD fuel`.	This is the computational assumption of the paper, stated on the fixed instance of the setting and over the reduction's solver family only.

Two definitions in `SDitH/Assumptions.lean` are candidate readings of the paper's §5.3 assumptions and are not consumed by the final theorem: `SDHardness` (planted uniform syndrome decoding with a runtime-accounted solver family) and `sharedOracleModel` (the joint ROM/ICM experiment). The final theorem uses the shared hash-and-cipher model directly through the game's definition, so the ideal model is a definition rather than a hypothesis; the planted distribution is not used because nothing relates the key generation's seed-expanded systematic instance to it.

The structural hypotheses of the final theorem are not assumptions in the cryptographic sense but facts about the setting that the theorem takes as given: the verifier's middle is the candidate middle (`hmid`), the challenge encoding is the Gray-row ψ map (`hencode` , `hSencode`), the BLC hash chain's encoding is injective on corrections and on the (plain, base) pair (`hencoding` , `hlinesEnc`), the witness sampler returns solutions (`hsample`), there is at least one higher coefficient (`hhigher`), and the budgets are ordered (`hcommit` , `hverify` , `hvc` , `hq`). The two encoding facts cannot be discharged in-model because the specification has no concrete serialization instance; see section 7.

Component results

Below the reduction, each of the submission's algorithms has a typed specification and proofs about it. The table gives, per algorithm, the specification module under `SDitH/Spec/` , the main proof modules under `SDitH/Proofs/` , and what is proved; the fourth column is what the literal text does not allow one to state. The full per-item disposition, including the unnumbered subroutines, tables and claims, is `audit/DISPOSITIONS.md` ; the per-module detail is `audit/COVERAGE.md` .

Alg.	Paper item	Spec module	Proof modules (main)	Proved	Not statable from the literal text
1	ExpandWitness	`WitnessExpansion`	`WitnessExpansion` , `WitnessConstraints` , `IntegerSamplingTrace`	Executed semantics at the cipher-stream boundary; returned weight and length; trace agreement with Algorithm 6 over four bytes per attempt	PRG block size; the undefined bindings it reads
2	Key generation	`KeyGeneration`	`KeyGeneration` , `PartialKeyGeneration`	Seed-draw law, ExpandH correspondence, literal syndrome assignment, generated-solution SD condition; the literal routine returns no key (`B` , `x_A` undefined)	`B` , `x_A` , systematic-matrix convention, raw codecs
3	Signing	`Signing`	`Signing` , `SharedSigning` , `SharedGameInvariant` , `CandidateSchemeChain` (18 modules)	Prepared records, staged cipher opening, signature provenance, fuel stability; the honest candidate signer's commitment program and its coupling in the reduction	Initialization, codecs; correspondence of the candidate signer to the literal algorithm
4	Verification	`SignatureComposition`	`SignatureComposition` , `CoefficientCompression` , `SharedScheme` , `SharedSchemeChain`	Executed program on the shared world (reconstruction, Hashaux, middle, Hashlines, suffix); forked accepted attempts determine a seed vector up to collisions	Codecs, Algorithm-12 line 21; the candidate verifier's explicit middle
5	TweakSalt	`DomainSeparation`	`DomainSeparation`	Bit-level computation and recoverability	nothing
6	PRG	`PRG`	`PRG`	Counter loop, parse shape, least block count, prefix/reader/sampler correspondence	Block-size denominator; present-salt/absent-tweak case. Refuted: positive Category-III calls cannot satisfy the printed complete-Parse assignment

Alg.	Paper item	Spec module	Proof modules (main)	Proved	Not statable from the literal text
7	BAVC.Commit	`BAVC`	`BAVC`, `BAVCBinding`, `BAVCBindingROM`	Full construction; tree recursion; serialization injectivity; computable collision finder; binding bounds in the ROM/ICM	Hiding is in the reduction only, through `DataClose` and `LeafHide`
8	BAVC.Open	`BAVCOpening`	`BAVCOpening`	Queue computation, termination, memory safety, padded length, trace	Reveal correctness relative to a corrected reconstruction
9	BAVC.Reconstruct	`BAVCReconstruction`	`BAVCReconstruction`, `BAVCPrefill`, `CandidateBAVC`, `CandidateCompleteness`	Full staged body; equal-root disagreement reduces to a hash or seed collision; executed cipher programs. Refuted: the Category-I opening is rejected for every `nodes` binding (`Final.commitment_discrepancies`)	`tree` versus the undeclared `nodes`; padding versus bounds
10	BLC.Commit	`BLCArithmetic`	`BLCArithmetic`, `HiddenShareTranslation`, `JointPads`, `RevealedBijection`, `HonestViewUniform` (18 modules)	Seed tree, sampling, Gray folding and corrections, typed hashes, consistency matrix, lines; concrete primitives with finite PRG correspondence; honest data uniform under a uniform hidden vector	Structured serialization, Category-III parse
11	BLC.OpenRandomEvaluation	`PartialBLCRetry`	`BLCRetry`, `ChallengeEncoding`, `CipherBLCRetry`	Retry with full staged cipher expansion; provenance and stable returns	Overflow policy; the retry bound is the named `hretry`
12	BLC.RecomputeEvaluation	`BLCRecompute`	`BLCHashChain`, `BLCReconstructionChain`, `BLCSeedExtraction`, and the reduction's 296 modules	Prefix and suffix execution; chain collision bounds; unique seed vector from two chains; the affine fold; the slope-consistency bound; the candidate middle; the composed reduction	Line 21 (ill-typed; the binary reading is `Candidate…`); the seed array's binding; the omitted witness-correction field of `Hashlines`
13, 14	PIOP Format	`PIOPArithmetic`	`PIOPArithmetic`	Parsing, mask computation, degree, evaluation	nothing
15, 16	PIOP UnitaryCheck	`PIOPArithmetic`	`UnitaryProbability`, `PIOPEncodingDiscrepancies`	Arithmetic, evaluation, quadratic bound. Refuted: the degree-one annotation	nothing further
17, 18	PIOP MuxTree	`PIOPArithmetic`	`PIOPDegree`, `MuxLinearity`, `MuxSelect`	Recurrence, evaluation, degree; with unitary selectors the tree reads the selected leaf	Pseudocode indexing and witness-order conventions
19, 20	PIOP Run	`PIOPArithmetic`	`PIOPComposition`, `PIOPMaskSplit`, `HashedPads`, `TopCoefficient`, `HashedView`, `DetailedPIOPChallenges`	Composition, evaluation, degree, constant-term identity, uniform batching law, comparison bound. Refuted: literal Category-I completeness (`Final.literal_piop_not_complete`), Table 3's dimensions (`Final.table3_dimension_mismatch`)	Intended witness correspondence; concrete challenges
21	BatchLinearEquations	`Batching`	`Batching`, `ConcreteEncodings`, `BatchingProbability`	Residual identity, padded embedding injectivity in all Table-2 fields, exact 2^-λ cancellation probability	Concrete challenge integration (`εBatch` in the reduction)

The concrete primitives are also specified and checked: SHAKE (Keccak with the FIPS refinements), AES-128 and Rijndael-256-256 as keyed permutations with test-vector certificates, and the concrete PRG's counter loop (`Spec/SHAKE`, `Spec/Keccak`,

`Spec/Rijndael`, `Spec/CipherPrimitives`, `Spec/PRG`). They are deterministic bodies; no randomness or indistinguishability claim is made about them, which is why the ideal model is a definition and its relation to these primitives is a review item.

The mathematics around the scheme is covered where the paper states something checkable: Table 2′s field representations (rings, bases, irreducibility, injectivity), the binary unitary polynomial and equation (6), the §5.1 SD-to-RSD regularization law and permutation correctness, the §6 variant tables, the key-codec capacities and Tables 4 and 5. Where the paper′s claim is false, the refutation is a theorem: the introductory PIOP soundness bound in d, the density floor and sign formulas of the parameter selection, the Category-III parse shape, and the prose tree-size/grinding disagreement.

What is not covered and why

The reduction (Track A of `PLAN.md`) is complete modulo the named assumptions of section 5. Everything else that a complete formalization of the submission would contain is blocked on one of four things, and each item below names which.

Item	Blocker	What exists today
Correctness of the scheme (honest signatures verify)	Refuted, then a decision: the literal PIOP check and the literal reconstruction reject honest inputs (`Final.literal_piop_not_complete`, `Final.commitment_discrepancies`), so no correctness theorem exists for the literal scheme; a corrected target has to be chosen.	`CandidateCompleteness` proves completeness of the candidate BAVC reconstruction; the candidate scheme is a reading for the parts the reduction needs, not a full corrected specification.
Category III	Scope: the reduction′s settings fix the cipher′s key and block width to λ, and the Category-III truncated layout has 192-bit seeds in 256-bit blocks, so it is outside the settings′ type.	`truncatedLayout` is defined and its output-uniformity is proved (`IdealCipherSeed`, continuation 122); the chain collision bounds cover the Category-III capacities. Extending the settings to a separate block width is a refactor of the reduction′s modules.

Item	Blocker	What exists today
Serialization and the encoding hypotheses	Decision: the source leaves the byte encodings, field order and transcript boundaries undefined, so `Spec` has no concrete `Serialization` instance and `hencoding`, `hlinesEnc` cannot be discharged.	The hash-chain injectivity is proved for the abstract `encoding` structure (`BLCHashChain`); fixed-width serialization injectivity is proved for the BAVC.
Hardness distribution	Modelling decision: `InstanceHard` is on the fixed instance; `SDHardness` is on the planted uniform distribution; nothing relates the seed-expanded systematic matrix of Algorithm 2 to that distribution, and there is no runtime model.	The regularization law and permutation step of §5.1 are proved (`SDReduction`, `SDReductionGame`); the printed event equality and conditional-uniqueness estimate are refuted and cannot be assumed.
Correspondence of the candidate scheme to Algorithms 3, 4 and 12	Decision: Algorithm 12′s line 21 is ill-typed and its seed array unbound; the `Hashlines` payload omits the witness correction; `B` and `x_A` are undefined.	The literal routines are specified up to the point where they read an undefined element, and the candidate readings are separately named; `audit/HUMAN_REVIEW.md` records each decision point.
PRG, challenge and batching idealizations	Decision taken: kept as assumptions (`DataClose`, `LeafHide`, `hretry`, `SimulatorTermBounds5`).	`PRGUniform` (`PRGHypothesis`) is the named predicate under which the data-level step from a uniform seed goes through (`DataSimulation`, `HiddenShareStep`); reversing the decision for any one hypothesis is a local proof project.
Variants (F256, TCitH, simple) and §4.1 greedy selection	Separate scope; the source gives outlines, and the density claims are refuted.	Tables 7 to 10, the scaled arithmetic and the comparison bound are proved (`ScaledWitness`, `ScaledArithmetic`, `VariantParameters`).

Item	Blocker	What exists today
Benchmarks and attack-cost estimates	Not provable: transcribed data.	`ReportedBenchmarks` records Table 6 as data; no theorem certifies it.

The decisions the project needs from a person before more proof work can start are therefore: which hardness distribution the instance should be related to and whether a runtime model is wanted; a reading of the byte encodings; whether the settings should be extended for Category III; and whether any of the four idealizations should be adopted after all. `audit/DISPOSITIONS.md` gives a proposed disposition for every source item as the input for that review.